



Implementation Considerations for Improving Data Integrity in Normalized Relational Databases

Tennyson X. Chen, Martin D. Meyer, Nanthini Ganapathi

Research Computing Division

RTI International¹

3040 Cornwallis Road

Research Triangle Park, NC 27709

Telephone: 01-(919)-990-8450

Emails: chen@rti.org, mmeyer@rti.org, Nanthini@rti.org

ABSTRACT

Using vertical decomposition method to normalize relational database has been well illustrated and understood. The purpose of relational decomposition is to remove data redundancy and prevent data anomalies. Up to the Boyce-Codd normal form (BCNF), decompositions are performed based on functional dependency at a relation's attribute level. However, since data redundancies can also appear between attributes that are interrelated in forms other than functional dependency, decomposition method fails to remove some very basic data anomalies. Consequently, we can still observe data anomalies in high level normal forms such as BCNF, 4NF and 5NF. After introducing the data anomalies that may occur in high level normal forms, this paper discusses some practical approaches on how to eliminate or reduce such anomalies in order to improve data integrity.

Keywords

Functional Dependency, Decomposition, Data Integrity, Functional Interdependency, Normalization.

1. INTRODUCTION

Functional dependency, which can be defined as follows, is an important concept in relational database normalization.

Given a relation R, attribute Y of R is functionally dependent on attribute X of R (noted as $X \rightarrow Y$) if and only if each X-value in R has associated with it precisely one Y-value in R (at any time) [7].

Whenever $X \rightarrow Y$ holds in R, X is called the determinant.

Functional dependency can cause data redundancies and anomalies. Using vertical decomposition method based on functional dependency to normalize relational databases has been very well illustrated in [3], [7], [11], as well as other relational database literature. Decomposition method is

wildly applied in today's relational database design.

Based on functional dependency among attributes², different levels of normal form are defined in [3] as follows,

A relation R is in first normal form (1NF) if and only if all underlying domains contain atomic values only.

A relation R is in second normal form (2NF) if and only if it is in 1NF and every non-key attribute is fully dependent on the primary key (no Partial Dependency).

A relation R is in third normal form (3NF) if and only if it is in 2NF and every non-key attribute is non-transitively dependent on the primary key (no Transitive Dependency).

A relation R is in Boyce-Codd Normal Form (BCNF) if and only if every determinant is a candidate key in R.

BCNF is considered to be a high level normal form in many database designers' view. However, functional dependency is just one of the interrelationships between attributes that can cause data anomalies. A recent study in [2] indicates that data redundancies similar to those caused by functional dependency can appear between attributes that are not functionally dependent on each other. If two attributes in a relation are not **functionally independent**, meaning they are interrelated in any way, data anomalies may still occur in BCNF or even higher level normal forms.

Functional Interdependency is defined in [2] as,

Given a relation R, attribute X of R is functionally independent from attribute Y of R (noted as $X \nrightarrow Y$) if and only if, for any instance $x_i \in \text{Domain}(X)$ and any instance $y_j \in \text{Domain}(Y)$, instance (x_i, y_j) is always valid for attribute set (X, Y) in R.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ACMSE '09 March 19-21, 2009, Clemson, SC, USA.

©2009 ACM 978-1-60558-421-8/09/03 ...\$10.00

¹ RTI International is a trade name of Research Triangle Institute.

² Throughout the paper, when a relation's "attribute" is mentioned, it refers to one single attribute or a set of attributes.

In other words, if two attributes are functionally independent, there can be no restrictions on how the values in these two attributes can appear in the relation.

If two attributes are not functionally independent, the term Functional non-Independency or Functionally non-Independent is used in this paper to describe the general interrelationship between these attributes.

A simple example of data anomalies caused by Functional non-Independency is as follows,

Example 1.

Relation Persons(ID, Title, Last Name, Gender), where ID is a unique identification number, is used to record personal information. A few tuples are shown in Table 1.

Table 1. Relation Persons

ID	Title	Last Name	Gender
101	Mr	Cooper	M
102	Mr	Smith	M
103	Dr	Armstrong	F
104	Dr	Davis	M
105	Ms	Taylor	F

Since ID is a unique number for each tuple, we can designate this attribute as the primary key, and the relation is now in BCNF.

However, we can easily find some basic data redundancies in this relation, because attributes Title and Gender are not functionally independent. Instances (Mr, F) and (Ms, M) should not be allowed in attribute set (Title, Gender). Nevertheless, these instances may occur in the relation when a tuple is inserted or updated. Therefore, insertion or update anomaly exists in this relation. Since the value “Dr” in Title can be associated with two values (M or F) in Gender, these two attributes are not functionally dependent on each other, so we cannot use conventional decomposition methods to further normalize this relation.

In this example, data anomalies are caused by a domain subset of Gender, {M, F}, being functionally dependent on a domain subset of Title, {Mr, Ms}. The nature of these anomalies is very similar to that of functional dependency. However, since we cannot establish functional dependency on these two attributes in their entirety, decomposition methods cannot be used to eliminate the data anomalies.

As a matter of fact, the data anomalies shown in all the examples that we have included in this paper cannot be eliminated automatically by traditional vertical decomposition methods. Therefore, the solutions in the Template Dependency discussed in [9], Subset Dependency in [10], and the Sixth Normal Form (6NF) in [5] bear little relevance since these solutions all rely on decomposition methods.

The Domain Key Normal Form (DK/NF) defined in [6] would eliminate functional non-independency in a DK/NF relation. However, there are no direct normalization

algorithms to achieve DK/NF. The horizontal decomposition method proposed in [6] can be used in certain circumstances. But this method will leave a relation fragmented and is practically very difficult to implement in most cases.

While complete theoretical solutions to eliminate data anomalies caused by functional non-independency among attributes are still being sought, this paper offers three practical strategies we can use to limit such anomalies. In our own practical experience with designing and implementing large relational databases, we have broadly adopted these methods and found them effective in preserving data integrity when dealing with specific problems in a database.

The methods and tools we are proposing, especially the conventional RDBMS tools used to implement the second and third methods, are not new techniques in relational database implementation. Rather, the unique contribution of this paper is that we explain how to systematically apply conventional tools to prevent specific data anomalies that are unfortunately omitted by the traditional normalization process. Also, this paper presents a formal description of the circumstances under which these troublesome anomalies can occur.

2. PRACTICAL APPROACHES FOR IMPROVING DATA INTEGRITY

In this section, we discuss some practical approaches that can be used to deal with data anomalies in high level normal forms. We will illustrate each method with an example, and discuss each method’s strengths and shortfalls in different situations. All examples in this paper are normalized into BCNF to start with. In fact, since the relations in these examples also satisfy the Multi-valued Dependency and Join Dependency requirements that respectively pertain to 4th Normal Form (4NF) and 5th Normal Form (5NF), they are all in 5NF, despite the data anomalies we will show.

Method 1: Redesign Relation Schema

After having normalized a database design by identifying functional dependencies among attributes, and after performing vertical decompositions as suggested by the traditional normalization process, database designers should take additional steps to analyze functional non-dependencies and try to eliminate them if possible. In certain situations, changing the schema design can be effective in removing data anomalies, as we see in Example 2.

Example 2.

A household study on television viewing habits conducted by a research company identifies each household with a unique identification number. This study also records the total number of TV sets in every house and the number of TV sets placed in bedrooms in the same house.

In order to store the information, we can naturally design a relation TVSET (HouseID, TotalTV, BedroomTV), where

HouseID is a unique household identification number, and TotalTV and BedroomTV store the total number of TV sets and that are placed in bedrooms, respectively. In this relation, the domains are as follows,

HouseID: {1, 2, 3, ...},
TotalTV: {0, 1, 2, ...}, and
BedroomTV: {0, 1, 2, ...}.

The following functional dependencies hold,

HouseID → TotalTV, and
HouseID → BedroomTV.

If we designate HouseID as the primary key, then relation TVSET is in BCNF. However, attributes TotalTV and BedroomTV are not functionally independent because in any tuple, the value of TotalTV should always be equal to or greater than that of BedroomTV. Instance (1, 2) is not valid for the attribute set (TotalTV, BedroomTV). For reasons similar to that of Example 1, this relation has insertion and update anomalies.

There are no decomposition methods that can prevent this kind of data anomaly. Rather, after examining the semantic requirements of this relation, especially the relationship between TotalTV and BedroomTV, we should consider changing the relational design as follows,

TVSET_1(HouseID, non-BedroomTV, BedroomTV)

In this relation, attribute TotalTV is replaced by non-BedroomTV where TotalTV = non-BedroomTV + BedroomTV. The domain of attribute non-BedroomTV is {0, 1, 2, ...}. This new relation TVSET_1 is in BCNF but it does not have the data anomaly described above because non-BedroomTV and BedroomTV are functionally independent of each other. In addition, for any tuple, the value of TotalTV can be derived from the sum of non-BedroomTV and BedroomTV, so no information is lost due to this redesign.

As we can see from this example, both relations TVSET and TVSET_1 are in BCNF. But one is better at handling data anomalies than the other. We can implement TVSET_1 by changing the schema of TVSET, instead of by decomposing the relation. Therefore, we can conclude that decomposition methods alone are not enough to remove data anomalies when implementing a relational database. Analyzing functional independencies should also be considered in addition to traditional database normalization procedures.

The main advantage of using the schema alteration method is that this process can completely eliminate functional non-independency at the design level. A relation schema that has been successfully redesigned with this method will have none of the data anomalies discussed above. This process is straightforward and does not require any other database system tools to further enhance data integrity. Hence, the practical utility of the method is hard to improve on.

The primary disadvantage of this method is that not all relation schemas can be redesigned in this manner. We can only apply this method when the semantic requirements are compatible. For instance, we cannot use this approach to deal with the problem shown in Example 1. The second drawback is that we need to be careful when redesigning a relation schema in order to preserve the meaning of a relation and prevent loss of useful information. Another practical consideration is that changing a relation schema may affect the processes by which data are collected and entered into the database. And these processing changes might prove costly if they are not made early in the database design phase.

Method 2: Enforce Referential Integrity

When changing a schema design is not possible, some standard RDBMS tools can be helpful in preventing or reducing data anomalies. One approach which is based on the use of standard tools involves the enforcement of referential integrity and the use of lookup tables. First we build a lookup table that holds all possible values of an attribute or attribute set on the main table. Then we enforce referential integrity on the main table to this lookup table. This mechanism ensures that the values in the underlying attributes in the main table are validated whenever a tuple is inserted or updated.

Referential Integrity is defined in [4] as,

Loosely, Referential Integrity is the rule that no database is allowed to contain any unmatched foreign key values. More precisely, let FK be some foreign key in some referencing relation R2; let K be the corresponding key in the corresponding referenced relation R1; and let K' be obtained from K as explained in the entry for foreign key. Then referential integrity requires that there never be a time at which an FK value exists in R2 that doesn't simultaneously exist as the K' value for some (necessarily unique) tuple in R1.

Traditionally, referential integrity is enforced to ensure that relations in a database are properly connected. We can use the same technique to limit data anomalies, as we see in Example 3.

Example 3.

Table 3.1 shows a few sample tuples from a relation Addresses (ID, Street No., Street Name, City, State, Zip Code), where ID is a unique number that can be used to identify a tuple in the relation.

Table 3.1 Relation Addresses

ID	Street No.	Street Name	City	State	Zip Code
1	101	1ST St.	Cleveland	AL	35049
2	233	2ND St.	Cleveland	AL	35049
3	125	A Dr.	Birmingham	AL	35215
4	256	B Dr.	Hoover	AL	35216
5	54	C Dr.	Birmingham	AL	35216

The rules among City, State, and Zip Code are as follows,

- The same city name can appear in different states.

- b. Although some zip codes are uniquely associated with their corresponding cities and states like tuples 1 and 2, some zip codes are used in multiple cities like tuples 4 and 5.
- c. A city may have multiple zip codes like tuple 3 and 5.
- d. Although it rarely happens, a zip code can be used in multiple states.

Based on these rules, we know that functional dependency does not hold among City, State, and Zip Code. Therefore, we can designate ID as the primary key and relation Addresses is in BCNF. However, data redundancy exists in this relation as we can see from tuples 1 and 2. The identical City, State, and Zip Code values appear multiple times. This is because a subset of the zip codes can uniquely determine a subset of cities and states. This is the same problem we saw in Example 1. This could lead to data anomalies when tuples are inserted or updated. Furthermore, general decomposition methods cannot prevent these data anomalies.

Applying referential integrity techniques by using a lookup table that holds all possible valid Zip Code, City, State values can help to limit this kind of data anomaly. Table 3.2 shows a few tuples of relation Zipcodes (Zip Code, City, State). In this relation, all three attributes are part of the primary key.

Table 3.2 Relation Zipcodes

Zip Code	City	State
35049	Cleveland	AL
35215	Center Point	AL
35215	Birmingham	AL
35216	Hoover	AL
35220	Center Point	AL

When implementing the database, we can set up a foreign key constraint so that the attributes in Addresses are linked to the corresponding attributes in Zipcodes. A generic SQL syntax for defining referential integrity for this model is given below.

```
ALTER TABLE [dbo].[Addresses]
ADD CONSTRAINT [FK_addresses_Zipcodes]
FOREIGN KEY ([ZipCode],
              [City],
              [State])
REFERENCES [dbo].[Zipcodes]
           ([ZipCode],
           [City],
           [State])3
```

If we enforce referential integrity in this way, every time a tuple in Addresses is inserted or updated, the City, State, Zip Code values are validated through the relation Zipcodes. An insertion or update operation in Addresses will succeed only when a matched tuple is found in Zipcodes. Since relation

Zipcodes only contains valid combinations of Zip Code, City, and State, invalid entries cannot possibly appear in Addresses via tuple insertion or update. Therefore, by keeping clean records in Zipcodes, we remove the data anomalies in relation Addresses.

In cases where the values in the lookup table need to be modified, the SQL standard provides the following referential integrity cascading actions for updates and deletions.

- No Action: This is the default action. It causes updates and deletions to the lookup table to fail if there is a foreign key reference.
- Cascade: The update of records may cascade to any foreign key records that refer to the affected rows in the lookup table.
- Set Null: The updates or deletions to the lookup table may cause the affected rows in the foreign key table to be set null.
- Set Default: Same as “Set Null” above except that the foreign key could be set to a predefined default value instead of null.

Different RDBMS systems like SQL Server, Oracle, MySQL, and MS Access support some or all of the above actions. These actions are further explained in [1].

The main advantages of defining referential integrity in a database are improved data quality and data consistency. Rather than allowing free input of data, only the values in the lookup table are allowed in the main table. The data are kept valid and intact by the database itself. Also, since referential integrity is declared at the design level, there is no need for custom programming, so there is reduced potential for introduction of unintended programming errors.

There are some disadvantages to this method. First of all, as we can see in our example, we have not really addressed the issue that a subset of cities and states is functionally dependent on a subset of zip codes. It is possible to have data anomalies in relation Zipcodes. Although zip code 35049 should only be valid for Cleveland, AL, the same zip code can be inserted into the relation with other city and state values. If the content in the lookup table is not valid, the whole mechanism breaks down. Secondly, there are operational concerns. This method requires an increased level of processing to validate data. Another possible problem is that using “cascade” deletions and updates may cause inadvertent loss of data without the knowledge of the user. Finally, if the lookup values require constant updates or user manipulations, this method will not only increase the processing burden, it may even cause potential data errors in the main table as well.

This method works best in situations where there are finite, pre-defined and standard lookup values, such as the zip code data, that do not require constant user manipulations. We can also use this method to alleviate the problem explained in Example 1, by predefining the finite values for (Title, Gender) in a lookup table.

³ When writing SQL statements in this paper, we are using SQL Server 2000 syntax.

When there are infinite possible combinations of lookup values, the use of lookup tables is not suitable. This leads us to the third method.

Method 3: Apply Check Constraints

In situations where we cannot use referential integrity, we can consider applying another RDBMS tool called “check constraint”. We illustrate this in Example 4.

Example 4.

A Human Resource department keeps salary information for all employees in a company. Each employee is identified by a unique numerical identification number. An employee belongs to a salary grade that is identified with an alpha character. Each salary grade is associated with a minimum salary value. The actual annual salary of an employee must be greater than the minimum salary of the grade that he/she is in.

The functional dependencies in this model are as follows,

EmployeeID $\rightarrow$ SalaryGrade,
EmployeeID $\rightarrow$ Salary,
SalaryGrade $\rightarrow$ MinimumSalary

BCNF can be achieved by designing two relations Employees(EmployeeID, SalaryGrade, Salary) and Grades(SalaryGrade, MinimumSalary).

Some example contents of these two relations are shown in Table 4.1 and 4.2. As we can tell, the third tuple in relation Employees contains a data anomaly, because the salary is lower than the minimum value of the corresponding grade. Unfortunately, nothing can prevent this tuple from being inserted or updated into this BCNF relation as designed.

Table 4.1. Relation Employees

EmployeeID	SalaryGrade	Salary
100	A	52,000
101	B	63,000
102	C	68,000

Table 4.2. Relation Grade

SalaryGrade	MinimumSalary
A	50,000
B	60,000
C	70,000

For each salary grade, only the minimum value is known. It is impractical to build a finite set of look up values. Therefore, we cannot use the referential integrity method to enforce data integrity in this case. Instead, we can consider using the check constraint method.

Traditionally, a RDBMS check constraint is designed to limit the domain scope of an attribute. As described in [8], by limiting the domain of an attribute to a set of allowable values, check constraints improve domain integrity. For example, when implementing relation Persons in Example 1, the data type of attribute Gender can only be defined as “string” in most of the commercial RDBMS. That means

this attribute can take any string value. If we want to limit the domain of this attribute to be {F, M}, we can define a check constraint as follows,

```
ALTER TABLE [dbo].[Persons]
ADD CONSTRAINT [CK_Persons_Gender]
CHECK ([Gender] = 'F' or [Gender] = 'M')
```

We can adopt this same technique to prevent data anomalies such as the one stated in Example 4, by building the following check constraint.

```
ALTER TABLE [dbo].[Employees]
ADD CONSTRAINT [CK_Employees_Salary]
CHECK ((([Grade] = 'A' and [Salary] >= 50000) or
([Grade] = 'B' and [Salary] >= 60000) or
([Grade] = 'C' and [Salary] >= 70000)))
```

After we implement this check constraint, any attempts to insert or update a tuple in Employees with an invalid salary grade, or with an out of range salary value, will cause the RDBMS to generate an error and the operation will not succeed. We can also easily use this same method to deal with the data anomaly described in Example 1.

The main advantages of the check constraint method are its simplicity and its ability to handle situations that the previous two methods cannot. In some instances, the check constraint method can also simplify the relation model. In Example 4, relation Grade may no longer be necessary if the business model does not utilize it for purposes other than limiting the salary values in relation Employees.

The main disadvantage of the check constraint method is that it is not easy to maintain. If we need to frequently change the information related to the Salary Grade in this example, we will need to constantly change the SQL code in the check constraint definition, which itself is a process that can generate errors. In addition, with this method, some useful information is hidden in SQL code rather than being held in table form. Therefore the information cannot be easily retrieved by using queries or with other standard RDBMS tools.

3. CONCLUSION

The traditionally defined normalization process of a relational database, which relies on vertical decomposition, is not enough to eliminate some basic data anomalies. This paper illustrates the fact that common data anomalies can indeed exist in high level normal forms. We have also provided three practical approaches for further refining the design of a normalized database. Each method has its own merits and shortcomings. Therefore, depending on the specific requirements at hand, a practitioner can consider one or more of these methods when implementing a database. We believe this extra step can greatly improve data integrity. We also believe that this paper can serve as a good educational discussion in identifying various methods that are required in removing data anomalies, although some of the methods proposed are not necessary new. There are also other RDBMS tools, such as Rules

and Triggers, that we have not included in the discussion. We will leave the applications of these tools in preventing data anomalies to the interested readers.

4. ACKNOWLEDGEMENT

We offer our special thanks to Albert Bethke and Phil Cooley of RTI International for their support in making this paper possible. We would also like to thank our entire software development team that has given us wonderful ideas and tremendous inspiration in putting together the pieces. Last but not least, we are grateful to Dr. Teng Chen of University of Hawaii for proof reading the paper and giving us valuable advice.

REFERENCES

- [1] Blaha, M. Referential Integrity Is Important For Databases, Modelsoft Consulting Corp. On line: <http://www.odbms.org/download/007.02%20Blaha%20Referential%20Integrity%20Is%20Important%20For%20Databases%20November%202005.PDF>
- [2] Chen, T., Liu, S., Meyer, M., and Gotterbarn, D. An Introduction to Functional Independency in Relational Database Normalization. In *Conference Proceedings of the 45th ACM Southeast Conference*, pages 221-225, Winston Salem, NC, 23-24 March 2007.
- [3]. Date, C. J. An Introduction to Database Systems (8th Edition). Reading, Mass.: Addison-Wesley Pub. Co, 1999.
- [4]. Date, C. J. The Relational Database Dictionary. Sebastopol, CA: O'Reilly Media, Inc., 2006
- [5] Date, C. J., Darwen, H., and Lorentzos, N. Temporal Data & the Relational Model (First Edition). San Fransisco, CA: Morgan Kaufmann, 2002
- [6]. Fagin, R. A Normal Form for Relational Databases That Is Based on Domains and Keys. In *ACM Transactions on Database Systems (TODS)*, Volume 6 Issue 3, pages 387-415, 1981.
- [7]. Kroenke, M. D. Database Processing: Fundamentals, Design, Implementation. New York: Macmillan Pub. Co, 1992.
- [8]. Rankins, R., Bertucci P., Gallelli C., and Silverstein, A. T. Microsoft SQL Server 2005 Unleashed. Indianapolis, IN: Sams Publishing, 2007
- [9]. Sadri, F. and Ullman D. J. A Complete Axiomatization for a Large Class of Dependencies in Relational Databases. In *Conference Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing*, pages 117-122, Los Angeles, CA, 28-30 April 1980.
- [10]. Sagiv, Y. and Walecka S. Subset Dependencies as an Alternative to Embedded Multivalued Dependencies. In *Journal of the Association for Computing Machinery*, Volume 29:1, pages 103-117, 1982.
- [11]. Ullman D. J. Principles of Database Systems. Rockville, MD.: Computer Science Press, 1982